

程序设计实践

The Practice of Programming

排错(Debugging)

什么是BUG，什么是DEBUG

bug.

b. A defect or fault in a machine, plan, or the like. orig. *U.S.*

1889 Pall Mall Gaz. 11 Mar. 1/1 Mr. Edison, I was informed, had been up the two previous nights discovering 'a bug' in his phonograph—an expression for solving a difficulty, and implying that some imaginary insect has secreted itself inside and is causing all the trouble.

Oxford English Dictionary. 2nd Edition

First actual case of bug being found

9/9

0800 Antam started
1000 " stopped - antam ✓

1300 (032) MP-MC ~~1.582647000~~
 (033) PRO 2 2.130476415 (23) 4.615925059(-2)
 conck 2.130676415

Relays 6-2 in 033 failed special speed test
in relay " 10.00 test.

Relays changed

1100 Started Cosine Tape (Sine check)
1525 Started Multi-Adder Test.

1545  Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.

~~1630~~ Antam started.
1700 closed down.

Relay 2145
Relay 3370

如何对待BUG

首先，最好不要出BUG

采用良好的设计避免BUG。

采用良好的编程风格，编程习惯来避免BUG。

注意所采用的语言的编译器的告警，不要总是忽略它们。

其次，让BUG容易发现

设计良好的用户界面（日志，错误信息，查看命令）

边界条件的检查以及断言的使用

再次，即使出现BUG，也要自己先发现并解决

完善的单元测试用例

设计自动测试脚本

再再次，自己没有发现，也要尽可能让组织内部的测试人员发现

组织内要有完善的测试制度

最后，不可避免，发布给用户的程序还是会出BUG...

如何对待BUG

一般应用程序运行出现**BUG**，首先，要**99.99%**确信：

- 不会是硬件的问题导致
- 不会是网络传输的错误导致
- 不会是操作系统的**BUG**导致
- 不会是编译器的**BUG**导致
- 不会是标准库的**BUG**导致

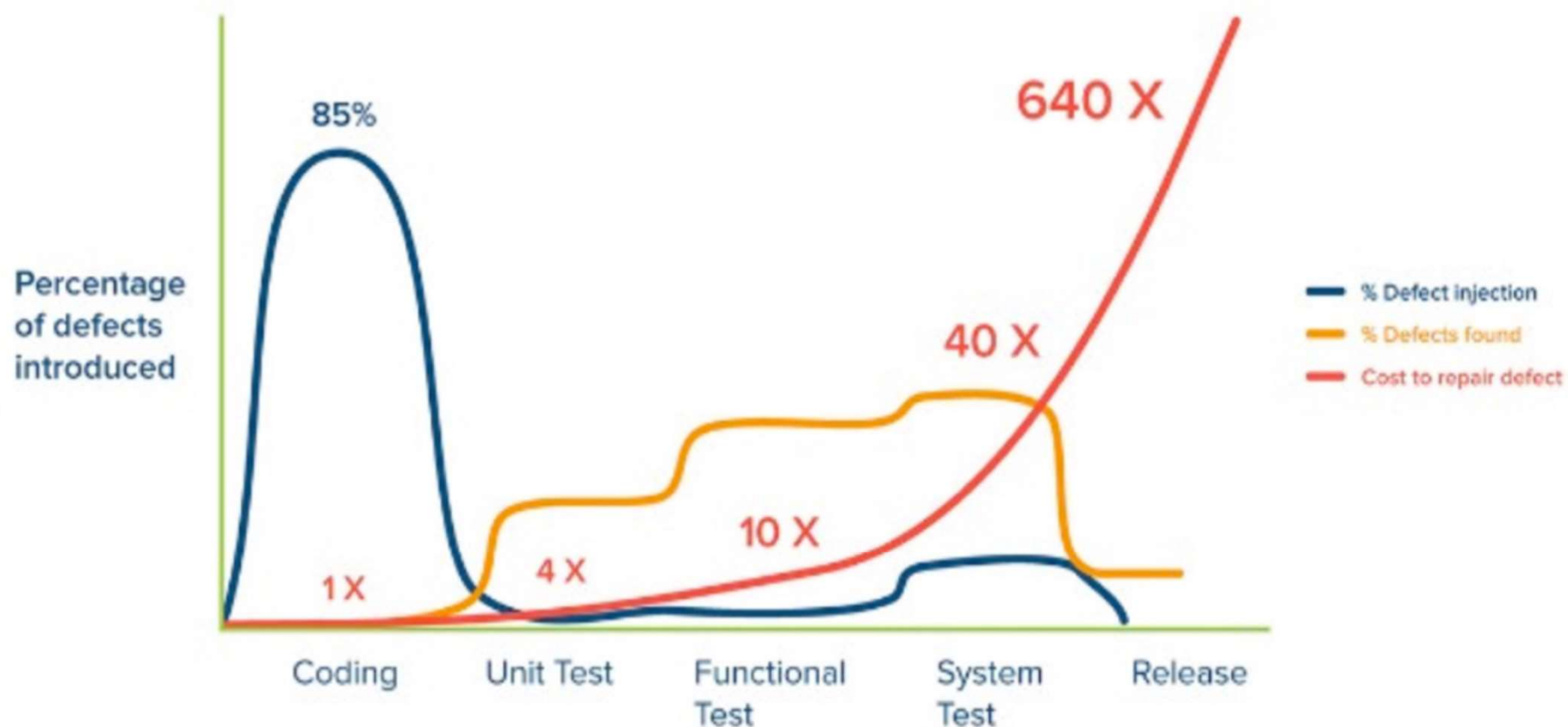
程序运行出现**BUG**，不知为何，**BUG**又消失了。但是实际上：

- **BUG**肯定仍然存在
- 而且这可能是一个比较难找的**BUG**
- 应该尽可能让这个**BUG**复现，以便解决它。

端正态度，努力从自己的代码找问题，尽早将**BUG**彻底解决掉。

注意： **BUG**越晚解决，解决它所需成本越高

BUG越晚解决，解决它所需成本越高



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

DEBUG工具(debugger)

大部分语言的IDE都集成有**debugger**，可以对程序执行单步跟踪，可以实时查看所有变量，但是，它们在处理一些较难的**BUG**的时候用途有限。

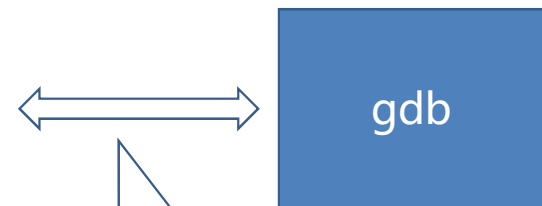
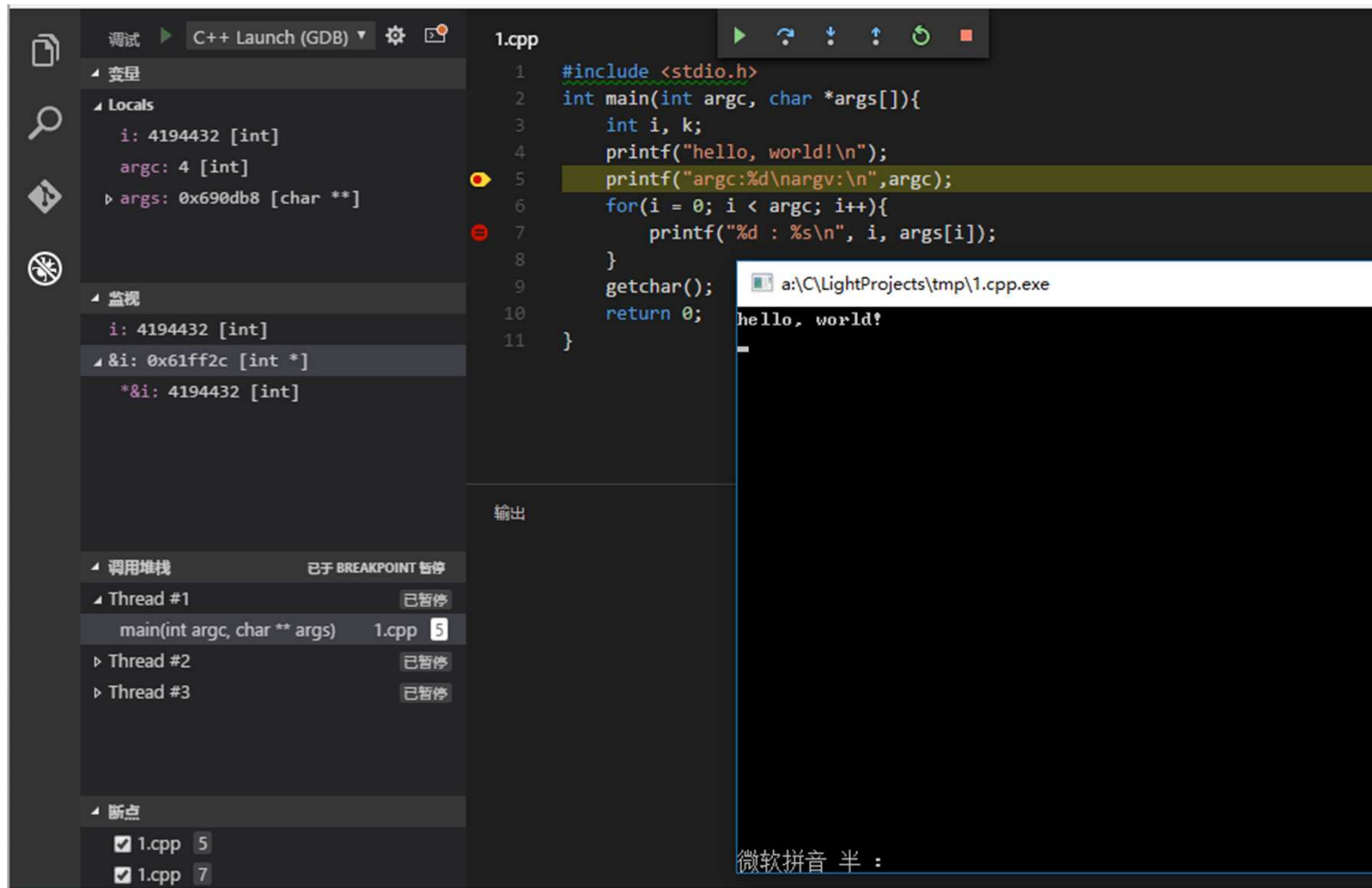
linux或者**unix**下都有基于命令行的调试工具**gdb**或者**dbx**，它功能上覆盖了IDE支持的所有功能，例如设置断点，单步执行，查看变量等。

gdb或者**dbx**经常用于查看程序崩溃时留下的**core**文件，可以方便地查看崩溃时的函数调用栈状态，指出崩溃在当前栈顶函数的哪一行。但是，如果指针的误用导致栈被严重破坏，这时候此工具就没有作用了。

程序执行遇到异常的时候，一般可以输出当前的堆栈信息有助于**BUG**的分析。

不过，工具解决的一般都是比较显而易见的**BUG**。很多难以解决的**BUG**，都是依靠在程序中增加打印语句的方法一点一点逻辑推理解决的。

DEBUG工具 - vscode



MI(Machine Interface)接口, 例如:

- -break-insert 添加断点
- -exec-run 启动程序
- -exec-next 单步跳过
- -exec-step 单步进入
- -var-create 创建变量监视
- -stack-list-frames 列出调用栈

DEBUG工具 - gdb

gdbtest.c

```
1 #include <stdio.h>
2 int main(int argc, const char* args[]){
3     int i, k;
4     printf("hello, world!\n");
5     printf("argc:%d\nargv:\n", argc);
6     for(i = 0; i < argc; i++){
7         printf("%d : %s\n", i, args[i]);
8     }
9     getchar();
10    return 0;
11 }
```

使用 **-g** 选项编译，可以使得可执行文件携带丰富的调试信息

```
$ g++ -o gdbtest -g gdbtest.c
```

编译之后，使用gdb 可执行文件名 开始debug

```
$ gdb gdbtest
...
(gdb) b 6
Breakpoint 1 at 0x400671: file gdbtest.c, line 6.
(gdb) r 111 222 333
Starting program: /home/hbzhang/test/gdbtest 111 222 333
hello, world!
argc:4
argv:

Breakpoint 1, main (argc=4, args=0x7fffffff258) at gdbtest.c:6
6         for(i = 0; i < argc; i++){

(gdb) p i
$1 = 0
(gdb) p argc
$2 = 4
(gdb)
```

设置第6行为断点 (Breakpoint)

携带参数111 222 333运行 (Run) 这个程序

打印 (Print) 变量i的值

打印 (Print) 变量argc的值

DEBUG工具 - gdb

有问题的
gdb.C

```
1 #include <stdio.h>
2 int inputInt()
3 {
4     int i;
5     scanf("%d", i);
6     return i;
7 }
8 int main()
9 {
10     int a = inputInt();
11     printf("%d\n", a);
12     return 0;
13 }
```

编译

```
$ g++ -g gdb.C
```

运行, 出现问题

```
$ ./a.out
123
段错误 (core dumped)
```

查看生成的core文件

```
$ ls core.*
core.44508
```

```
$ gdb a.out core.44508
GNU gdb (GDB) Red Hat Enterprise Linux (7.2-75.el6)
...
(gdb) where
#0  0x000000357ae56fbb in _IO_vfscanf_internal (s=<value optimized out>, format=0x40074a "",
argptr=0x7fffffa355630, errp=0x0) at vfscanf.c:1826
#1  0x000000357ae65cdc in __scanf (format=<value optimized out>) at scanf.c:33
#2  0x0000000000400620 in inputInt () at gdb.C:5
#3  0x0000000000400632 in main () at gdb.C:10
(gdb)
```

DEBUG工具 - java的printStackTrace

```
1 import java.io.*;
2 public class Hello {
3     public static int div(int a, int b) throws Exception{
4         return a / b;    // 此处没有判断b是否为0, 如果参数b代入0, 可能会导致除0错误
5     }
6     public static void main(String args[]){
7         try{
8             int i = div(100, 0);
9         }
10        catch(Exception e){
11            e.printStackTrace();
12        }
13    }
14 }
```

编译, 运行

```
$ javac Hello.java
$
$
$ java Hello
java.lang.ArithmeticException: / by zero
    at Hello.div(Hello.java:4)
    at Hello.main(Hello.java:8)
```

栈信息显示:

- main函数 (第8行), 调用了
- div函数 (第4行) 出现了除0错误

DEBUG工具 - c/c++的backtrace

NAME

backtrace, backtrace_symbols, backtrace_symbols_fd - support for application self-debugging

SYNOPSIS

```
#include <execinfo.h>
```

```
int backtrace(void **buffer, int size);
```

```
char **backtrace_symbols(void *const *buffer, int size);
```

```
void backtrace_symbols_fd(void *const *buffer, int size, int fd);
```

DESCRIPTION

backtrace() returns a backtrace for the calling program, in the array pointed to by buffer. A backtrace is the series of currently active function calls for the program. Each item in the array pointed to by buffer is of type void *, and is the return address from the corresponding stack frame. The size argument specifies the maximum number of addresses that can be stored in buffer. If the backtrace is larger than size, then the addresses corresponding to the size most recent function calls are returned; to obtain the complete backtrace, make sure that buffer and size are large enough.

Given the set of addresses returned by backtrace() in buffer, backtrace_symbols() translates the addresses into an array of strings that describe the addresses symbolically. The size argument specifies the number of addresses in buffer. The symbolic representation of each address consists of the function name (if this can be determined), a hexadecimal offset into the function, and the actual return address (in hexadecimal). The address of the array of string pointers is returned as the function result of backtrace_symbols(). This array is malloc(3)ed by backtrace_symbols(), and must be freed by the caller. (The strings pointed to by the array of pointers need not and should not be freed.)

DEBUG的一般步骤

1. 了解清楚**BUG**的表现
2. 获取这个**BUG**的线索
3. 查阅相关版本的代码修改
4. 在自己的开发环境复现**BUG**
5. 在复现**BUG**的开发环境下分析
6. 二分法缩小**BUG**的范围
7. 不能复现的(Non-reproducible) **BUG**的处理

1. 了解清楚BUG的表现

BUG的表现，包括如下几个方面：

出现BUG的前置条件（主机、操作系统、程序版本、配置等等）

出现BUG时的输入或操作

出现BUG之后的现象（输出不正确？死循环？异常退出？）

BUG发现的时机，影响对BUG表现的了解

如果是自己发现的错误，自己直接可以了解。

如果是组织内测试人员或运维人员发现的BUG，也比较容易了解到BUG的表现，但不如前者。

如果是用户投诉，了解BUG表现则相对困难。

所以，我们应该让BUG尽早被发现。

了解这个BUG表现的普遍性和特殊性

以前是否出现过类似的BUG？

主机，操作系统，版本这些条件是否特殊？

是偶然发生？还是频繁发生？还是必然发生？

2. 获取这个BUG的线索

BUG发生前后这段时间的日志

基于良好的接口设计，理应包含完善的日志文件，但是：

也许怀疑有问题的地方并没有记录日志（后悔中，为什么这里当时不加个日志呢？）

BUG发生时，也许有用的日志开关并未打开

也许日志太多，报告BUG的时候已经被冲掉

UNIX或者Linux系统下，如果发生异常退出，获取core文件

理想情况下，使用dbx或者gdb查看core文件，可以定位出问题的文件以及行号，但是：

某些情况下退出，并不会产生core文件

也许用户设置了禁止core文件产生

有一些指针越界导致的异常，堆栈被破坏得很严重，core文件无法提供有用的信息

使用维护命令获取尽可能多的信息

基于良好的接口设计，理应有命令可以查看当前的程序状态，但是：

严重的BUG导致程序退出，则现场就会被破坏，所有信息都已经被初始化。

BUG报告时间太晚，当前的状态无法提供有用的信息。

也许你关心的地方，并没有对应的维护命令（后悔中，为什么这里当时不加一个命令呢？）

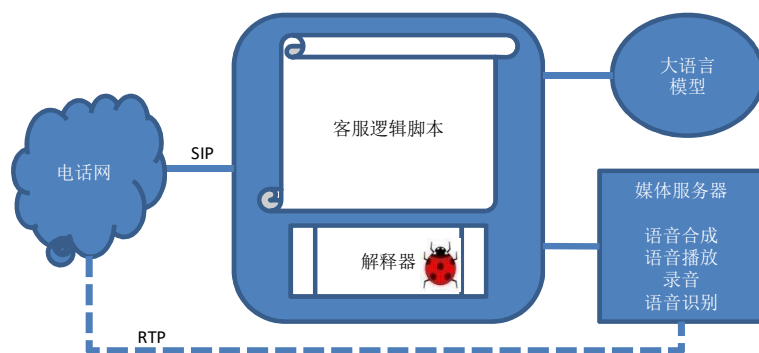
3. 查阅相关版本的代码修改

基于良好的习惯，所有所有代码的都应被版本管理工具妥善的管理

如果版本n出现的BUG，可以diff版本n与版本n-1的差别，仔细阅读这些差别



4. 在自己的开发环境复现BUG



开发环境往往无法与实际运行环境一致。需要设计各种测试桩尽可能模拟实际运行环境。



5. 在复现BUG的开发环境下分析

- 可以在有所怀疑的地方，任意增加日志，增加断言，增加命令。
- 尽可能认真地增加这些东西，让它们以后还有用。

天哪！我加上一行日志输出，BUG就不见了！
去掉日志BUG又出来了！

- 上面的问题不要奇怪，因为指针导致的问题，任何不可思议的现象都可能出现。
- 既然BUG消失了，那么刚才这个环境就是一个不能复现BUG的环境，毫无用处。
- 恢复到以前，重新修改。

6. 二分法缩小BUG的范围

(1-100行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(1-50行脚本 + 解释器) 执行10000次 => 没有内存泄漏

(51-100行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(51-75行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(51-63行脚本 + 解释器) 执行10000次 => 没有内存泄漏

(64-75行脚本 + 解释器) 执行10000次 => 发现内存泄漏

范围缩小之后，只有12行脚本，就有内存泄露，那么问题必然在这12行脚本涉及到的代码中。

7.不能复现的(Non-reproducible) BUG

确实有些BUG难以复现，或者需要很大的成本才能复现。尝试做这些事情：

1、使用工具扫描代码（例如splint），对工具给出的可疑结果认真检查。

2、回顾自己的经验教训，例如：

- 检查是否有没有初始化的变量？是否有需要返回值的函数没有返回值？

- 检查是否有错误的指针应用？

 - 函数返回了栈里的指针？

 - 指针重复删除？

 - 使用了删除指针指向的数据？

- 程序是否涉及多线程？

 - 是否你使用的库启动了多线程你不知道？

 - 是否有未加锁的共享资源访问？

 - 是否使用了线程不安全的函数？

3、把你的BUG，你的代码讲给你周围的人听。

